

The Anatomy of an Agent Harness

A deep dive into what Anthropic, OpenAI, Perplexity and LangChain are actually building. Covering the orchestration loop, tools, memory, context management, and everything else that transforms a stateless LLM into a capable agent.

You've built a chatbot. Maybe you've wired up a ReAct loop with a few tools. It works for demos. Then you try to build something production-grade, and the wheels come off: the model forgets what it did three steps ago, tool calls fail silently, and context windows fill up with garbage.

The problem isn't your model. It's everything around your model.

LangChain proved this when they changed only the infrastructure wrapping their LLM (same model, same weights) and jumped from outside the top 30 to rank 5 on TerminalBench 2.0. A separate research project hit a 76.4% pass rate by having an LLM optimize the infrastructure itself, surpassing hand-designed systems.

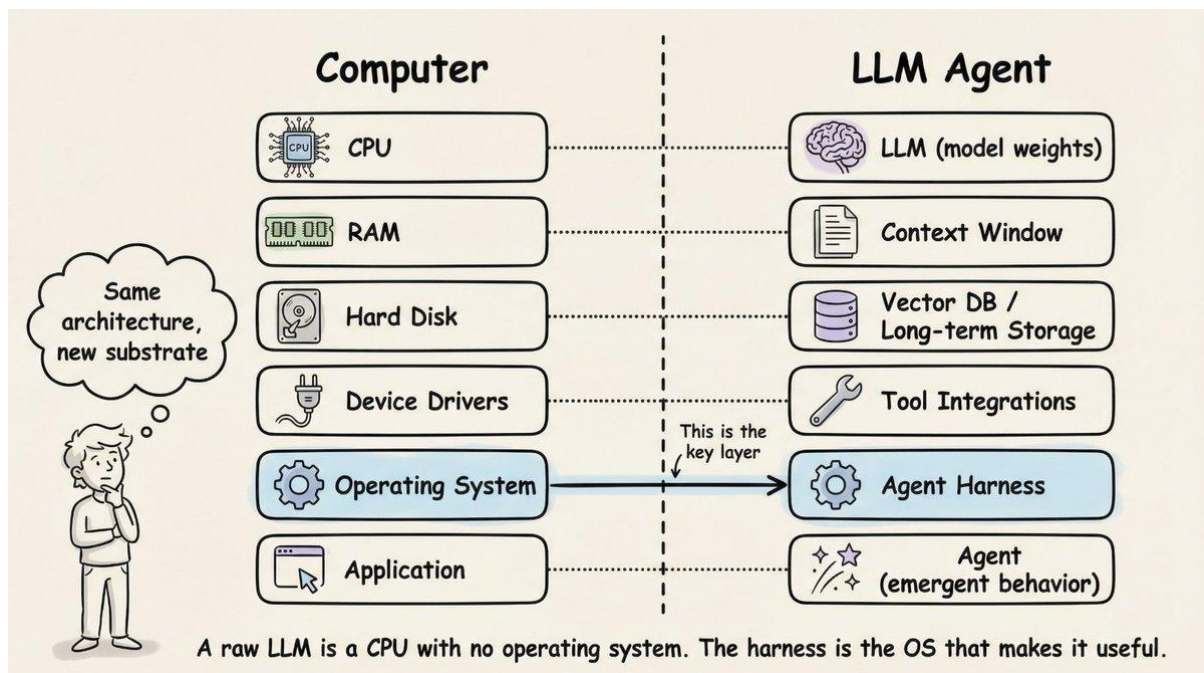
That infrastructure has a name now: **the agent harness**.

What Is the Agent Harness?

The term was formalized in early 2026, but the concept existed long before. The harness is the complete software infrastructure wrapping an LLM: orchestration loop, tools, memory, context management, state persistence, error handling, and guardrails. Anthropic's Claude Code documentation puts it simply: the SDK is "**the agent harness that powers Claude Code**." OpenAI's Codex team uses the same framing, explicitly equating the terms "**agent**" and "**harness**" to refer to the **non-model infrastructure** that makes the LLM useful.

I really liked the canonical formula, from LangChain's Vivek Trivedy: "**If you're not the model, you're the harness.**"

Here's the distinction that trips people up. The "agent" is the emergent behavior: the goal-directed, tool-using, self-correcting entity the user interacts with. The harness is the machinery producing that behavior. When someone says "I built an agent," they mean they built a harness and pointed it at a model.



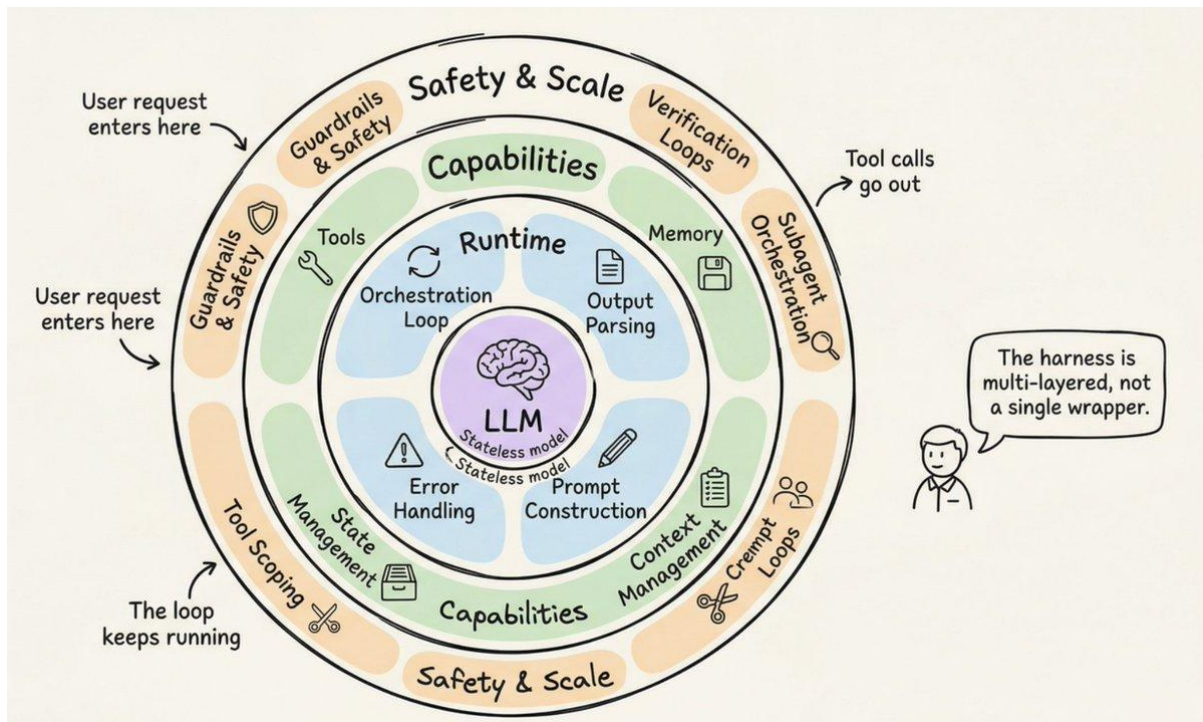
Beren Millidge made this analogy precise in his 2023 essay "[Scaffolded LLMs as Natural Language Computers](#)." A raw LLM is a CPU with no RAM, no disk, and no I/O. The context window serves as RAM (fast but limited). External databases function as disk storage (large but slow). Tool integrations act as device drivers. The harness is the operating system. As Millidge wrote: **"We have reinvented the Von Neumann architecture"** because it's a natural abstraction for any computing system.

Three Levels of Engineering

Three concentric levels of engineering surround the model:

- **Prompt engineering** crafts the instructions the model receives.
- **Context engineering** manages what the model sees and when.
- **Harness engineering** encompasses both, plus the entire application infrastructure: tool orchestration, state persistence, error recovery, verification loops, safety enforcement, and lifecycle management.

The harness is not a wrapper around a prompt. It is the complete system that makes autonomous agent behavior possible.



The 12 Components of a Production Harness

Synthesizing across Anthropic, OpenAI, LangChain, and the broader practitioner community, a production agent harness has twelve distinct components. Let's walk through each one.

1. The Orchestration Loop

This is the heartbeat. It implements the Thought-Action-Observation (TAO) cycle, also called the ReAct loop. The loop runs: assemble prompt, call LLM, parse output, execute any tool calls, feed results back, repeat until done.

Mechanically, it's often just a while loop. **The complexity lives in everything the loop manages**, not the loop itself. Anthropic describes their runtime as a "dumb loop" where all intelligence lives in the model. The harness just manages turns.

2. Tools

Tools are the agent's hands. They're defined as schemas (name, description, parameter types) injected into the LLM's context so the model knows what's available. The tool layer handles registration, schema validation, argument extraction, sandboxed execution, result capture, and formatting results back into LLM-readable observations.

Claude Code provides tools across six categories: file operations, search, execution, web access, code intelligence, and subagent spawning. OpenAI's Agents SDK supports function tools (via [@function tool](#)), hosted tools (WebSearch, CodeInterpreter, FileSearch), and MCP server tools.

3. Memory

Memory operates at multiple timescales. **Short-term memory** is conversation history within a single session. **Long-term memory** persists across sessions: Anthropic uses [CLAUDE.md](#) project files and

auto-generated [MEMORY.md](#) files; LangGraph uses namespace-organized JSON Stores; OpenAI supports Sessions backed by SQLite or Redis.

Claude Code implements a three-tier hierarchy: a lightweight index (~150 characters per entry, always loaded), detailed topic files pulled in on demand, and raw transcripts accessed via search only. A critical design principle: **the agent treats its own memory as a "hint" and verifies against actual state before acting.**

4. Context Management

This is where many agents fail silently. The core problem is context rot: **model performance degrades 30%+ when key content falls in mid-window positions** (Chroma research, corroborated by Stanford's "Lost in the Middle" finding). Even million-token windows suffer from instruction-following degradation as context grows.

Production strategies include:

- **Compaction:** summarizing conversation history when approaching limits (Claude Code preserves architectural decisions and unresolved bugs while discarding redundant tool outputs)
- **Observation masking:** JetBrains' Junie hides old tool outputs while keeping tool calls visible
- **Just-in-time retrieval:** maintaining lightweight identifiers and loading data dynamically (Claude Code uses grep, glob, head, tail rather than loading full files)
- **Sub-agent delegation:** each subagent explores extensively but returns only 1,000 to 2,000 token condensed summaries

Anthropic's context engineering guide states the goal: find the **smallest possible set of high-signal tokens** that maximize likelihood of the desired outcome.

5. Prompt Construction

This assembles what the model actually sees at each step. It's hierarchical: system prompt, tool definitions, memory files, conversation history, and the current user message.

OpenAI's Codex uses a strict priority stack: server-controlled system message (highest priority), tool definitions, developer instructions, user instructions (cascading [AGENTS.md](#) files, 32 KiB limit), then conversation history.

6. Output Parsing

Modern harnesses rely on native tool calling, where the model returns structured `tool_calls` objects rather than free-text that must be parsed. The harness checks: are there tool calls? Execute them and loop. No tool calls? That's the final answer.

For structured outputs, both OpenAI and LangChain support schema-constrained responses via Pydantic models. Legacy approaches like `RetryWithErrorOutputParser` (which feeds the original prompt, the failed completion, and the parsing error back to the model) remain available for edge cases.

7. State Management

LangGraph models state as typed dictionaries flowing through graph nodes, with reducers merging updates. Checkpointing happens at super-step boundaries, enabling resume after interruptions and time-travel debugging. OpenAI offers four mutually exclusive strategies: application memory, SDK sessions, server-side Conversations API, or lightweight `previous_response_id` chaining. Claude Code takes a different approach: **git commits as checkpoints and progress files as structured scratchpads**.

8. Error Handling

Here's why this matters: **a 10-step process with 99% per-step success still has only ~90.4% end-to-end success**. Errors compound fast.

LangGraph distinguishes four error types: transient (retry with backoff), LLM-recoverable (return error as `ToolMessage` so the model can adjust), user-fixable (interrupt for human input), and unexpected (bubble up for debugging). Anthropic catches failures within tool handlers and returns them as error results to keep the loop running. Stripe's production harness caps retry attempts at two.

9. Guardrails and Safety

OpenAI's SDK implements three levels: input guardrails (run on first agent), output guardrails (run on final output), and tool guardrails (run on every tool invocation). A "tripwire" mechanism halts the agent immediately when triggered.

Anthropic separates permission enforcement from model reasoning architecturally. The model decides what to attempt; the tool system decides what's allowed. **Claude Code gates ~40 discrete tool capabilities independently**, with three stages: trust establishment at project load, permission check before each tool call, and explicit user confirmation for high-risk operations.

10. Verification Loops

This is what separates toy demos from production agents. Anthropic recommends three approaches: rules-based feedback (tests, linters, type checkers), visual feedback (screenshots via Playwright for UI tasks), and LLM-as-judge (a separate subagent evaluates output).

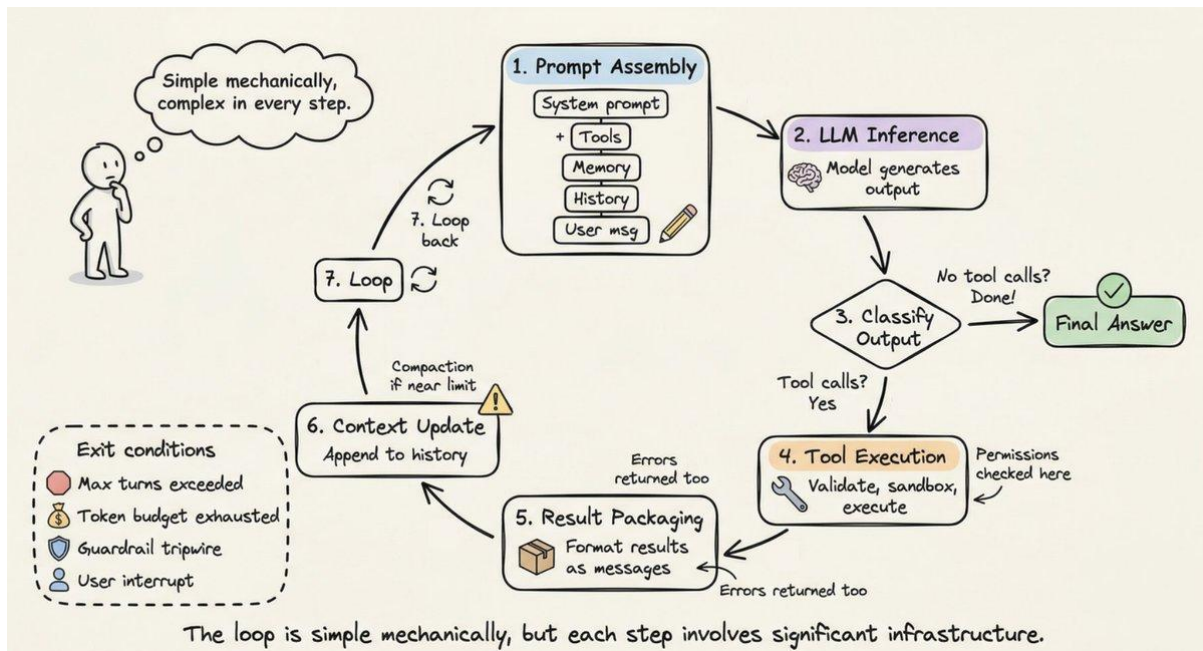
Boris Cherny, creator of Claude Code, noted that **giving the model a way to verify its work improves quality by 2 to 3x**.

11. Subagent Orchestration

Claude Code supports three execution models: Fork (byte-identical copy of parent context), Teammate (separate terminal pane with file-based mailbox communication), and Worktree (own git worktree, isolated branch per agent). OpenAI's SDK supports agents-as-tools (specialist handles bounded subtask) and handoffs (specialist takes full control). LangGraph implements subagents as nested state graphs.

The Loop in Motion: A Step-by-Step Walkthrough

Now that you know the components, let's trace how they work together in a single cycle.



Step 1 (Prompt Assembly): The harness constructs the full input: system prompt + tool schemas + memory files + conversation history + current user message. Important context is positioned at the beginning and end of the prompt (the "Lost in the Middle" finding).

Step 2 (LLM Inference): The assembled prompt goes to the model API. The model generates output tokens: text, tool call requests, or both.

Step 3 (Output Classification): If the model produced text with no tool calls, the loop ends. If it requested tool calls, proceed to execution. If a handoff was requested, update the current agent and restart.

Step 4 (Tool Execution): For each tool call, the harness validates arguments, checks permissions, executes in a sandboxed environment, and captures results. Read-only operations can run concurrently; mutating operations run serially.

Step 5 (Result Packaging): Tool results are formatted as LLM-readable messages. Errors are caught and returned as error results so the model can self-correct.

Step 6 (Context Update): Results are appended to conversation history. If approaching the context window limit, the harness triggers compaction.

Step 7 (Loop): Return to Step 1. Repeat until termination.

Termination conditions are layered: the model produces a response with no tool calls, maximum turn limit is exceeded, token budget is exhausted, a guardrail tripwire fires, the user interrupts, or a safety refusal is returned. A simple question might take 1 to 2 turns. A complex refactoring task can chain dozens of tool calls across many turns.

[For long-running tasks spanning multiple context windows, Anthropic developed a two-phase "Ralph Loop" pattern:](#) an **Initializer Agent** sets up the environment (init script, progress file, feature list, initial git commit), then a **Coding Agent** in every subsequent session reads git logs and progress files to orient itself, picks the highest-priority incomplete feature, works on it, commits, and writes summaries. The filesystem provides continuity across context windows.

How Real Frameworks Implement the Pattern

	Claude Agent SDK	OpenAI Agents SDK	LangGraph	CrewAI	AutoGen
Loop	Dumb loop, smart model 	Runner class 	State graph 	Sequential / Hierarchical 	Conversation-driven 
State	Git commits 	4 strategies 	Typed dicts + checkpoints 	Task results 	Message history 
Multi-Agent	Fork / Teammate / Worktree 	Agents-as-tools + Handoffs 	Nested graphs 	Agent-Task-Crew 	5 orchestration patterns 
Philosophy	Thin harness, trust the model 	Code-first 	Graph-based control 	Role-based collaboration 	Conversation as protocol 

Same pattern, different bets on where control should live

Frameworks have converged on the same core pattern but diverge in philosophy.

Anthropic's Claude Agent SDK exposes the harness through a single `query()` function that creates the agentic loop and returns an async iterator streaming messages. The runtime is a "dumb loop." All intelligence lives in the model. Claude Code uses a Gather-Act-Verify cycle: gather context (search files, read code), take action (edit files, run commands), verify results (run tests, check output), repeat.

OpenAI's Agents SDK implements the harness through the Runner class with three modes: async, sync, and streamed. The SDK is "code-first": workflow logic is expressed in native Python rather than graph DSLs. The Codex harness extends this with a three-layer architecture: Codex Core (agent code + runtime), App Server (bidirectional JSON-RPC API), and client surfaces (CLI, VS Code, web app). All surfaces share the same harness, which is why "Codex models feel better on Codex surfaces than a generic chat window."

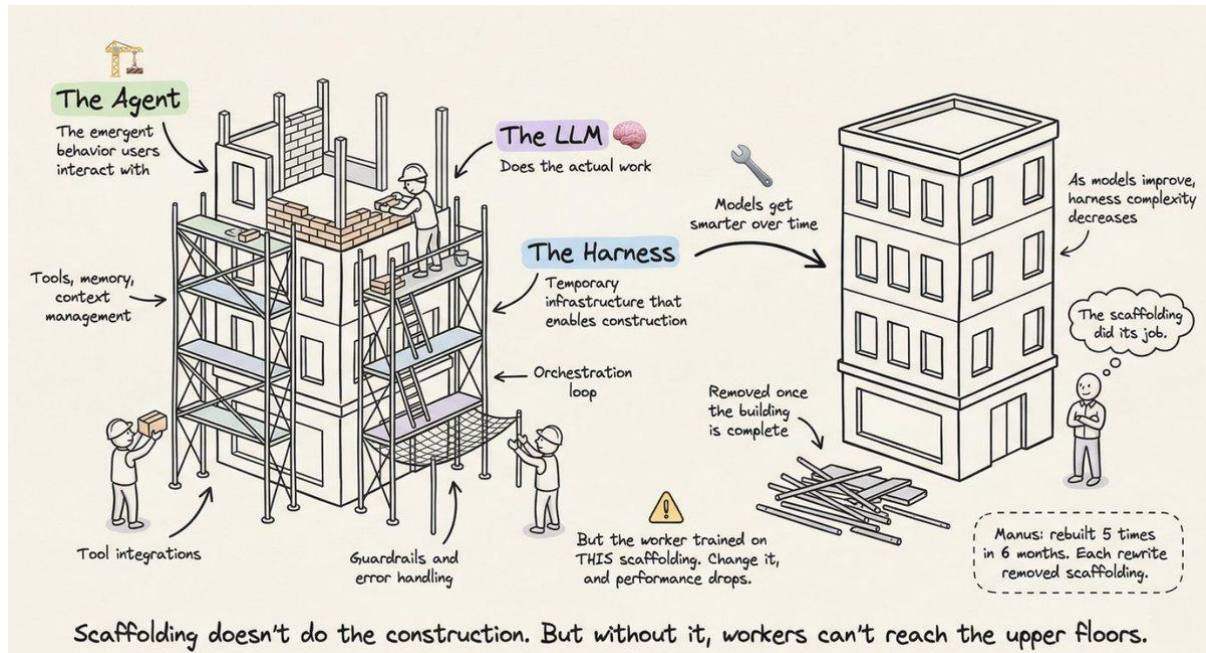
LangGraph models the harness as an explicit state graph. Two nodes (`llm_call` and `tool_node`) connected by a conditional edge: if tool calls present, route to `tool_node`; if absent, route to END. LangGraph evolved from LangChain's `AgentExecutor`, which was deprecated in v0.2 because it was hard to extend and lacked multi-agent support. LangChain's **Deep Agents** explicitly use the term "agent harness": built-in tools, planning (`write_todos` tool), file systems for context management, subagent spawning, and persistent memory.

CrewAI implements a role-based multi-agent architecture: Agent (the harness around the LLM, defined by role, goal, backstory, and tools), Task (the unit of work), and Crew (the collection of agents). CrewAI's Flows layer adds a "deterministic backbone with intelligence where it matters," managing routing and validation while Crews handle autonomous collaboration.

AutoGen (evolving into Microsoft Agent Framework) pioneered conversation-driven orchestration. Its three-layer architecture (Core, AgentChat, Extensions) supports five orchestration patterns: sequential, concurrent (fan-out/fan-in), group chat, handoff, and magentic (a manager agent maintains a dynamic task ledger coordinating specialists).

The Scaffolding Metaphor

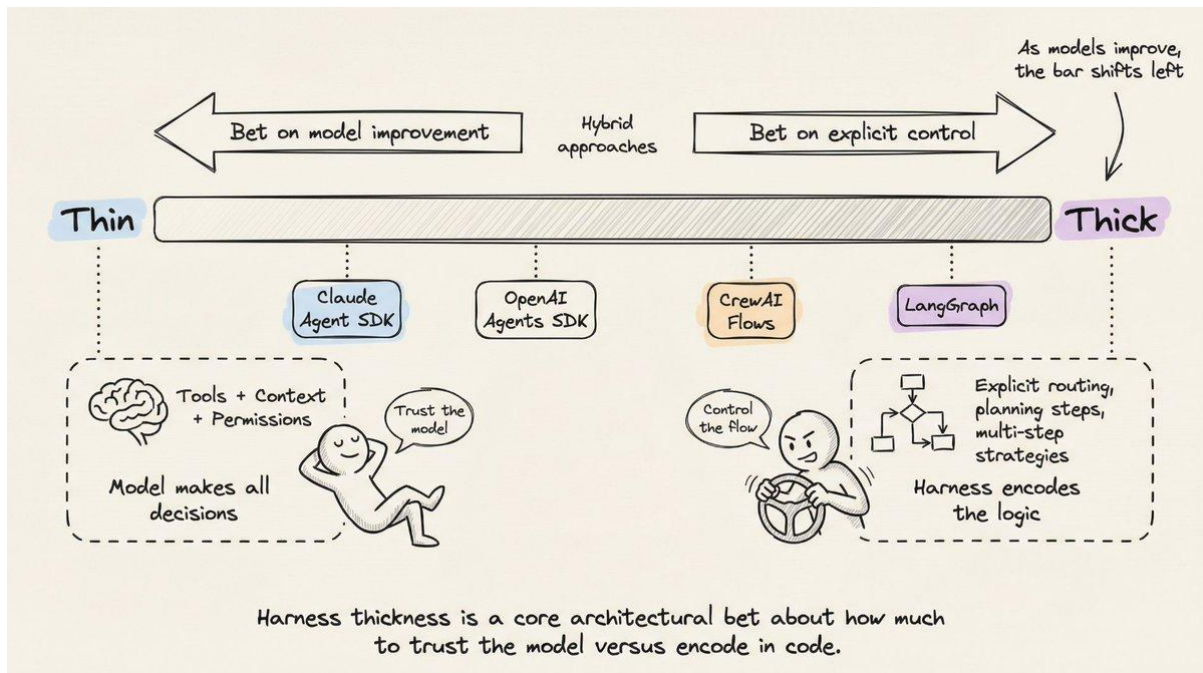
The scaffolding metaphor isn't decorative. It's precise. Construction scaffolding is temporary infrastructure that enables workers to build a structure they couldn't reach otherwise. It doesn't do the construction. But without it, workers can't reach the upper floors.



The key insight: scaffolding is removed when the building is complete. As models improve, harness complexity should decrease. Manus was rebuilt five times in six months, each rewrite removing complexity. Complex tool definitions became general shell execution. "Management agents" became simple structured handoffs.

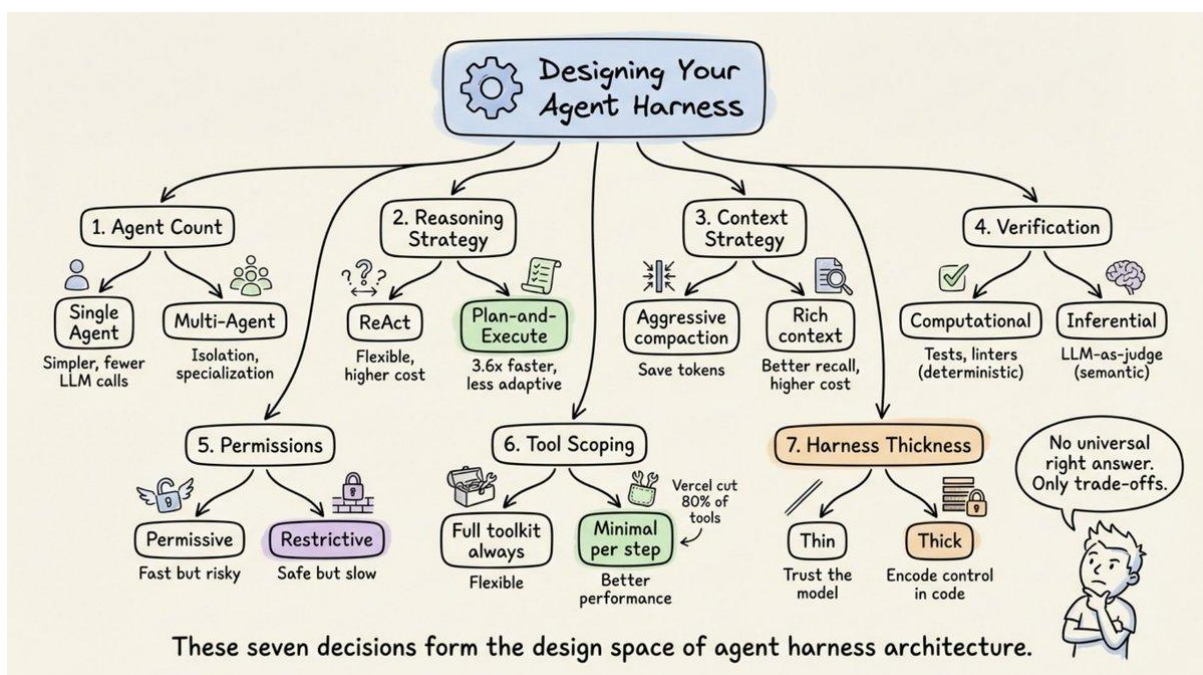
This points to the **co-evolution principle**: models are now post-trained with specific harnesses in the loop. Claude Code's model learned to use the specific harness it was trained with. Changing tool implementations can degrade performance because of this tight coupling.

The "future-proofing test" for harness design: if performance scales up with more powerful models without adding harness complexity, the design is sound.



Seven Decisions That Define Every Harness

Every harness architect faces seven choices:



- 1. Single-agent vs. multi-agent.** Both Anthropic and OpenAI say: maximize a single agent first. Multi-agent systems add overhead (extra LLM calls for routing, context loss during handoffs). Split only when tool overload exceeds ~10 overlapping tools or clearly separate task domains exist.
- 2. ReAct vs. plan-and-execute.** ReAct interleaves reasoning and action at every step (flexible but higher per-step cost). Plan-and-execute separates planning from execution. LLMCompiler reports a **3.6x speedup** over sequential ReAct.

3. **Context window management strategy.** Five production approaches: time-based clearing, conversation summarization, observation masking, structured note-taking, and sub-agent delegation. ACON research showed **26 to 54% token reduction while preserving 95%+ accuracy** by prioritizing reasoning traces over raw tool outputs.
4. **Verification loop design.** Computational verification (tests, linters) provides deterministic ground truth. Inferential verification (LLM-as-judge) catches semantic issues but adds latency. Martin Fowler's Thoughtworks team frames this as **guides** (feedforward, steer before action) versus **sensors** (feedback, observe after action).
5. **Permission and safety architecture.** Permissive (fast but risky, auto-approve most actions) versus restrictive (safe but slow, require approval for each action). The choice depends on deployment context.
6. **Tool scoping strategy.** More tools often means worse performance. Vercel removed **80% of tools** from v0 and got better results. Claude Code achieves **95% context reduction** via lazy loading. The principle: expose the minimum tool set needed for the current step.
7. **Harness thickness.** How much logic lives in the harness versus the model. Anthropic bets on thin harnesses and model improvement. Graph-based frameworks bet on explicit control. Anthropic regularly deletes planning steps from Claude Code's harness as new model versions internalize that capability.

The Harness Is the Product

Two products using identical models can have wildly different performance based solely on harness design. The TerminalBench evidence is clear: changing only the harness moved agents by 20+ ranking positions.

The harness is not a solved problem or a commodity layer. It's where the hard engineering lives: managing context as a scarce resource, designing verification loops that catch failures before they compound, building memory systems that provide continuity without hallucination, and making architectural bets about how much scaffolding to build versus how much to leave to the model.

The field is moving toward thinner harnesses as models improve. But the harness itself isn't going away. Even the most capable model needs something to manage its context window, execute its tool calls, persist its state, and verify its work.

The next time your agent fails, don't blame the model. Look at the harness.
